

Software Architecture & Design Patterns

Design Pattern

- Typical solutions to commonly occurring problems in software design.
- Pre-made blueprints that can be customized to solve a recurring design problem in code.

Describing Design Patterns

- Pattern Name and Classification
- Intent
- Also Known As
- Applicability
- Structure
- Participants

Describing Design Patterns(Cont.)

- Collaborations
- Consequences
- Implementation
- Sample Code
- Known Uses
- Related Patterns

Describing Design Patterns(Cont.)

- **Intent**

- What does the design pattern do?
- What is its rationale and intent?
- What particular design issue or problem does it address?

- **Motivation**

- A scenario that illustrates
 - a design problem and
 - how the class and object structures in the pattern solve the problem.

Describing Design Patterns(Cont.)

- **Structure** of classes shows each part of the pattern and how they are related.
- **Sample Code**
 - Code fragments that illustrates how you the pattern might be implemented.

Why Should We Learn Patterns

- Design patterns are a toolkit of tried and tested solutions to common problems in software design.
- Design patterns define a common language that can be used to communicate more efficiently.

Classification of Patterns

Scope	Class	Purpose		
		Creational	Structural	Behavioral
		Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

- The first criterion, called **purpose**, reflects what a pattern does.
- Patterns can have either creation, structural, or behavioral purpose.
- **Creational** patterns concern the process of object creation.
- **Structural** patterns deal with the composition of classes or objects.
- **Behavioral** patterns characterize the ways in which classes or objects interact and distribute responsibility.

- The second criterion, called **scope**, specifies whether the pattern applies primarily to classes or to objects.
- **Class patterns** deal with relationships between classes and their subclasses. These relationships are established through inheritance, so they are static— fixed at compile-time.
- **Object patterns** deal with object relationships, which can be changed at run-time and are more dynamic.
- Almost all patterns use inheritance to some extent. So the only patterns labeled "class patterns" are those that focus on class relationships. Note that most patterns are in the Object scope.

References

<https://refactoring.guru/>